ANÁLISIS DEL RETO 2 – CHOCOLATE SALES

ISIS-1225 Estructuras de Datos y Algoritmos – 2026-20 – Grupo G07

	Nombre	**Código**	**Correo Uniandes**
Est-1	Salomón González Lozada	202611956	s.gonzalez2@uniandes.edu.co
Est-2	Jeronimo Guerrero Maldonado	202610325	j.guerrero112@uniandes.edu.co
Est-3	Alan Guillermo Gonzalez	202612540	ag.gonzalez123@uniandes.edu.co

Asignación de requerimientos

Requerimiento	**Responsable**
Requerimiento 1 (individual)	Est-1: Salomón González Lozada
Requerimiento 2 (individual)	Est-2: Jeronimo Guerrero Maldonado
Requerimiento 3 (individual)	Est-3: Alan Guillermo Gonzalez
Carga de datos, Requerimientos 4, 5 y 6	Grupal

Notación

n: número total de pedidos cargados. k: número de pedidos asociados a la llave consultada (un país, un mes, una fecha, un producto o una combinación producto-país); siempre k ≤ n. m: número de pedidos que cumplen el filtro (m ≤ k). p: número de productos distintos. c: número de países distintos. Las operaciones put y get de las tablas de hash se toman como O(1) en promedio (factor de carga acotado y rehash amortizado). Los ordenamientos usan los algoritmos del curso: quick_sort (carga, Req 2 y Req 5), con costo O(x log x) en el caso promedio y $O(x^2)$ en el peor caso, y merge_sort (Req 1, Req 3 y Req 6), con costo O(x log x) en todos los casos.

Tablas de hash construidas durante la carga

Todas las tablas se crean en new_logic y se llenan en el mismo recorrido del archivo, de modo que cada pedido se inserta en todas ellas con un costo O(1) promedio. Los valores que son listas guardan referencias a los mismos pedidos de la lista principal, por lo que no se duplica información.

Tabla	**Tipo**	**Llave**	**Valor**	**Propósito**
pedidos_por_canal	Linear Probing (3 llaves, factor 0.5)	Channel	Contador (int)	Número de pedidos por canal en el reporte de carga.

pedidos_por_pais	Linear Probing (5 llaves, factor 0.5)	Country	array_list de pedidos	Acceso directo a los pedidos de un país (Req 2 y Req 5).
pedidos_por_mes	Linear Probing (24 llaves, factor 0.5)	"YYYY-MM"	array_list de pedidos	Acceso directo a los pedidos de un mes (Req 1).
pedidos_por_fecha	Separate Chaining (≈730 llaves, factor 4)	Order_Date	array_list de pedidos	Acceso directo a los pedidos de una fecha (Req 3).
pedidos_por_producto_pais	Linear Probing (producto × país, factor 0.5)	(Product, Country)	array_list de pedidos	Acceso directo a una combinación producto-país (Req 4).
pedidos_por_producto	Linear Probing (pocas llaves, factor 0.5)	Product	array_list de pedidos	Acceso directo a los pedidos de un producto (Req 6).

Criterio de selección del tipo de tabla: Linear Probing se usó cuando el número de llaves es pequeño y conocido (canales, países, meses, productos y combinaciones producto-país) y nunca se eliminan elementos; con factor de carga 0.5 casi no se forman grupos y cada entrada queda directamente en el arreglo. Separate Chaining se usó cuando el número de llaves es mayor o menos predecible (fechas), porque admite un factor de carga mayor que 1 sin degradarse bruscamente. Las tablas auxiliares que se crean dentro de los requerimientos (conteo y acumulados por producto o por país) son Linear Probing por la misma razón: tienen pocas llaves y no hay eliminaciones.

Carga de datos

Descripción

Se lee el CSV una sola vez. Por cada fila se crea el pedido con los tipos correctos, se agrega a la lista principal (array_list), se actualizan los acumulados del reporte (fecha más antigua y más reciente, menor y mayor Amount y conteo por canal) y se inserta el pedido en las tablas de hash. Al terminar se ordena la lista por Amount descendente para obtener los cinco primeros y los cinco últimos.

Entrada	Ruta del archivo CSV y catálogo vacío.
Salidas	Catálogo con la lista y las tablas de hash; reporte con tiempo, total de pedidos, pedidos por canal, pedidos de fecha más antigua y más reciente, de menor y mayor Amount, y primeros y últimos 5 por Amount.
Implementado (Sí/No)	Sí. Grupal.

Análisis de complejidad de cada uno de los pasos del algoritmo:

Pasos	**Complejidad**

Leer las n filas y crear cada pedido	O(n)
add_last en la lista principal (por pedido O(1))	O(n)
Actualizar acumulados del reporte (comparaciones y get/put en pedidos_por_canal)	O(n)
Insertar cada pedido en las tablas de hash (get/put/add_last)	O(n)
quick_sort de la lista por Amount descendente	O(n log n)
sub_list de los 5 primeros y 5 últimos	O(1)
TOTAL	***O(n log n)***

Análisis

Todo lo que se calcula por pedido es constante, por lo que el recorrido del archivo es O(n). El costo dominante es el ordenamiento para los cinco primeros y últimos, O(n log n). La carga se vuelve algo más lenta que en el Reto 1 porque construye las tablas, pero se paga una sola vez y permite que los requerimientos no recorran la lista principal. En las pruebas, la carga del 20 % (38.721 pedidos) tomó cerca de 565 ms y la del 100 % (193.608 pedidos) cerca de 3.515 ms: un crecimiento ligeramente mayor que lineal, coherente con O(n log n).

Requerimiento 1

Descripción

App > logic.py > sort_crit_boxes_shipped

```
258
259  def req_1(catalog, anio, mes, desc_min, desc_max):
260      """
261      Retorna el resultado del requerimiento 1: pedidos de un mes con
262      Discount_Pct dentro de un rango (inclusivo).
263
264      Complejidad: O(1) para el get del mes + O(k) para filtrar + O(k log k)
265      para ordenar con merge_sort, donde k = pedidos del mes (k <= n).
266      """
267      start = get_time()
268      filtrados, suma_amount, suma_precio, suma_cajas = filtrar_req1(catalog, anio, mes, desc_min, desc_max)
269      total = al.size(filtrados)
270      if total > 0:
271          promedio_precio = suma_precio / total
272          promedio_cajas = suma_cajas / total
273      else:
274          promedio_precio = 0
275          promedio_cajas = 0
276      filtrados = al.merge_sort(filtrados, criterio_req1)
277      end = get_time()
278      return {
279          "tiempo_ms": delta_time(start, end),
280          "total_pedidos": total,
281          "monto_total": suma_amount,
282          "promedio_precio": promedio_precio,
283          "promedio_cajas": promedio_cajas,
284          "pedidos": filtrados
```

Con el año y el mes se arma la llave "YYYY-MM" y se obtiene de pedidos_por_mes la lista de pedidos de ese mes. Se recorre esa lista y se conservan los pedidos con Discount_Pct dentro del rango, acumulando el monto total y las sumas de Price_per_Box y Boxes_Shipped para los promedios. Los filtrados se ordenan con merge_sort por Discount_Pct descendente, Amount descendente y Order_ID ascendente.

Entrada	Catálogo, año, mes, descuento mínimo y máximo.
Salidas	Tiempo, total de pedidos filtrados, monto total, promedios de Price_per_Box y Boxes_Shipped, y la lista ordenada (primeros y últimos 10 si son más de 20).
Implementado (Sí/No)	Sí. Implementado por Salomón González Lozada (Est-1).

Análisis de complejidad de cada uno de los pasos del algoritmo:

Pasos	**Complejidad**
get en pedidos_por_mes	O(1)
Recorrer los k pedidos del mes y filtrar por descuento	O(k)
Sumas y promedios	O(k)
merge_sort de los m filtrados	O(m log m)
TOTAL	***O(k + m log m)***

Análisis

La tabla reduce la búsqueda de n pedidos a los k del mes (aproximadamente n/24). El ordenamiento de los filtrados domina cuando el rango de descuento es amplio. Se eligió Linear Probing porque solo hay 24 meses en el dataset.

Requerimiento 2

Descripción

```python
def req_2(catalog, country, min_boxes_shipped, max_boxes_shipped):
    """
    Retorna el resultado del requerimiento 2: pedidos de un pais con
    Boxes_Shipped dentro de un rango (inclusivo).
    """
    start_time = get_time()

    filtered_list, mapa_productos, suma_marketing_spend, suma_amount, producto_mas_frecuente, numero_pedidos_produ
    # Numero total de pedidos que cumplieron el filtro
    pedidos_en_rango = al.size(filtered_list)
    # Numero de productos distintos en el rango
    cantidad_productos_distintos = mlp.size(mapa_productos)
    # Promedios de Marketing_Spend y Amount
    if pedidos_en_rango > 0:
        promedio_marketing_spend = suma_marketing_spend / pedidos_en_rango
        promedio_amount = suma_amount / pedidos_en_rango
    else:
        promedio_marketing_spend = 0
        promedio_amount = 0

    result = {
        'tiempo_de_ejecución': None,
        'numero_total_pedidos_cumplieron_filtro': pedidos_en_rango,
        'cantidad_productos_distintos_en_rango': cantidad_productos_distintos,
        'promedio_marketing_spend': promedio_marketing_spend,
        'promedio_amount': promedio_amount,
        'producto_frecuente': producto_mas_frecuente,
        'numero_de_pedidos': numero_pedidos_producto,
        'pedidos_filtrados': filtered_list,
    }
    end_time = get_time()
    result['tiempo_de_ejecución'] = delta_time(start_time, end_time)
    return result
```

Se obtiene de pedidos_por_pais la lista de pedidos del país. En un solo recorrido se filtran los pedidos con Boxes_Shipped dentro del rango; cada pedido filtrado se agrega a una array_list, se suman Marketing_Spend y Amount, y se actualiza su conteo en una tabla auxiliar producto → número de pedidos (Linear Probing). En el mismo paso se mantiene el producto más frecuente (mayor conteo; en empate, nombre alfabéticamente menor). Al final se ordena la lista por Boxes_Shipped ascendente, Order_Date descendente y Order_ID ascendente.

Entrada	Catálogo, país, número mínimo y máximo de cajas.
Salidas	Tiempo, total de filtrados, número de productos distintos (size de la tabla auxiliar), promedios de Marketing_Spend y Amount, producto más frecuente con su conteo y la lista ordenada (primeros y últimos 10 si son más de 20).
Implementado (Sí/No)	Sí. Implementado por Jeronimo Guerrero Maldonado (Est-2).

Análisis de complejidad de cada uno de los pasos del algoritmo:

Pasos	**Complejidad**
get en pedidos_por_pais	O(1)

Recorrer los k pedidos del país y filtrar por cajas	O(k)
Por cada filtrado: add_last, sumas, get/put en la tabla de productos y actualizar el más frecuente	O(1)
size de la tabla de productos y promedios	O(1)
quick_sort de los m filtrados	O(m log m)
TOTAL	***O(k + m log m)***

Análisis

El acceso por país evita recorrer los n pedidos, pero como solo hay cinco países cada lista es grande: en el 100 % Australia tiene 89.596 pedidos (cerca del 46 %) y Japan 11.289, así que el peor caso es Australia con un rango amplio de cajas, donde m ≈ k y domina O(k log k). La tabla de productos permite saber si un producto ya apareció en O(1) en lugar de una búsqueda lineal, y el producto más frecuente se obtiene sin recorrerla otra vez.

Requerimiento 3

Descripción

```python
def req_3(catalog, fecha, monto_min, monto_max):
    """
    Retorna el resultado del requerimiento 3: pedidos de una fecha con
    Amount dentro de un rango (inclusivo).

    Complejidad: O(1) para el get de la fecha + O(k) para filtrar + O(k log k)
    para ordenar con merge_sort, donde k = pedidos de ese dia.
    """
    start = get_time()
    filtrados, suma_amount, suma_marketing = filtrar_req3(catalog, fecha, monto_min, monto_max)
    total = al.size(filtrados)
    filtrados = al.merge_sort(filtrados, criterio_req3)
    end = get_time()
    return {
        "tiempo_ms": delta_time(start, end),
        "total_pedidos": total,
        "monto_total": suma_amount,
        "marketing_total": suma_marketing,
        "aporte_total": suma_amount - suma_marketing,
        "pedidos": filtrados
    }
```

Se obtiene de pedidos_por_fecha la lista de pedidos de la fecha dada. Se recorre filtrando por Amount dentro del rango y se acumulan el monto total, la inversión en mercadeo y el aporte comercial (Amount − Marketing_Spend). Los filtrados se ordenan con merge_sort por Amount descendente, aporte comercial descendente y Order_ID ascendente.

Entrada	Catálogo, fecha (YYYY-MM-DD), valor mínimo y máximo de Amount.
Salidas	Tiempo, total de filtrados, monto total, mercadeo total, aporte comercial total y la lista ordenada (primeros y últimos 10 si son más de 20).
Implementado (Sí/No)	Sí. Implementado por Alan Guillermo Gonzalez

	(Est-3).

Análisis de complejidad de cada uno de los pasos del algoritmo:

Pasos	**Complejidad**
get en pedidos_por_fecha	O(1)
Recorrer los k pedidos de la fecha y filtrar por Amount	O(k)
Sumas de monto, mercadeo y aporte	O(k)
merge_sort de los m filtrados	O(m log m)
TOTAL	***O(k + m log m)***

Análisis

Es el requerimiento que más se beneficia de la tabla: hay cerca de 730 fechas, así que k es del orden de n/730 y la consulta es prácticamente inmediata. Se usó Separate Chaining por la cantidad de llaves.

Requerimiento 4

Descripción

```python
def req_4(catalog, producto, pais):
    """
    Retorna el resultado del requerimiento 4: estadisticas de la combinacion
    Producto-Pais y los 2 pedidos de mayor Amount.

    Complejidad: O(1) para el get de la llave (producto, pais) + O(k) para
    recorrer el grupo una sola vez (sumas y top 2), donde k = pedidos de la
    combinacion. No hace falta ordenar: el top 2 se mantiene en una pasada.
    """
    start = get_time()
    candidatos = mlp.get(catalog["pedidos_por_producto_pais"], (producto, pais))
    suma_precio = 0
    suma_descuento = 0
    suma_marketing = 0
    suma_cajas = 0
    primero = None
    segundo = None
    total = 0
    if candidatos is not None:
        total = al.size(candidatos)
        for i in range(total):
            pedido = al.get_element(candidatos, i)
            suma_precio += pedido["price_per_box"]
            suma_descuento += pedido["discount_pct"]
            suma_marketing += pedido["marketing_spend"]
            suma_cajas += pedido["boxes_shipped"]
            # Se mantienen los dos mejores pedidos
            if primero is None or mejor_pedido_req4(pedido, primero):
                segundo = primero
                primero = pedido
            elif segundo is None or mejor_pedido_req4(pedido, segundo):
                segundo = pedido
    top_2 = al.new_list()
    if primero is not None:
        al.add_last(top_2, primero)
    if segundo is not None:
        al.add_last(top_2, segundo)
    if total > 0:
        promedio_precio = suma_precio / total
        promedio_descuento = suma_descuento / total
```

Con la tupla (producto, país) como llave se consulta pedidos_por_producto_pais y se obtiene directamente la lista de pedidos de esa combinación. Se recorre una vez sumando Price_per_Box, Discount_Pct, Marketing_Spend y Boxes_Shipped para los promedios, y manteniendo los dos pedidos de mayor Amount (desempate: menor Marketing_Spend y luego Order_ID ascendente).

Entrada	Catálogo, producto y país.
Salidas	Tiempo, total de pedidos, promedios de Price_per_Box, Discount_Pct, Marketing_Spend y Boxes_Shipped, y los dos pedidos de mayor Amount.
Implementado (Sí/No)	Sí. Grupal.

Análisis de complejidad de cada uno de los pasos del algoritmo:

Pasos	**Complejidad**

get en pedidos_por_producto_pais	O(1)
Recorrer los k pedidos de la combinación (sumas y top 2)	O(k)
Calcular promedios	O(1)
TOTAL	***O(k)***

Análisis

En el Reto 1 este requerimiento recorría toda la lista, O(n). En el Reto 2 la tabla entrega solo los pedidos de la combinación, y como el top 2 se mantiene durante el recorrido no hace falta ordenar. Por eso la consulta pasa de O(n) a O(k) y, en la comparación experimental entre ambos retos, su tiempo crece mucho menos con el tamaño del archivo.

Requerimiento 5

Descripción

```
def req_5(catalog, N, country, initial_date, final_date):
    """
    Retorna el resultado del requerimiento 5: los N productos con mayor
    recaudo para un pais en un rango de fechas (inclusivo).
    """
    start_time = get_time()

    mapa_productos, recaudo_total_filtro, pedidos_en_filtro = filtrado_req5(catalog, country, initial_dat

    # Registros de los productos en un array_list para poder ordenarlos
    values = mlp.value_set(mapa_productos)

    ordered_products = al.quick_sort(values, sort_crit_recaudo_total)

    if N >= mlp.size(mapa_productos):
        top = al.sub_list(ordered_products, 0, mlp.size(mapa_productos))
    else:
        top = al.sub_list(ordered_products, 0, N)

    # Promedios solo para los productos del top
    for i in range(0, al.size(top)):
        registro = al.get_element(top, i)
        registro['promedio_price_per_box'] = registro['suma_price_per_box'] / registro['num_pedidos']
        registro['promedio_discount_pct'] = registro['suma_discount'] / registro['num_pedidos']

    # Resultado para la vista
    result = {
        'tiempo_ejecucion': None,
        'num_total_pedidos_filtro': pedidos_en_filtro,
        'num_productos_diferentes': mlp.size(mapa_productos),
        'total_recaudo_pedidos_filtrados': recaudo_total_filtro,
        'productos_top': top

    }

    # Tiempo solo del requerimiento
```

Se obtiene de pedidos_por_pais la lista del país y se recorre filtrando por Order_Date dentro del rango (las fechas YYYY-MM-DD se comparan como texto). Por cada filtrado se suman el contador y el recaudo total, y se actualiza en una tabla auxiliar producto → registro (Linear Probing) los acumulados del producto: pedidos, cajas, recaudo, sumas de precio y descuento, y mercadeo. Después, value_set

entrega los registros en una array_list, que se ordena por recaudo descendente, cajas descendente y nombre ascendente; se toman los primeros N con sub_list y se calculan sus promedios.

Entrada	Catálogo, N, país, fecha inicial y fecha final.
Salidas	Tiempo, total de filtrados, número de productos distintos, recaudo total y, por cada producto del top N: pedidos, cajas, recaudo, Price_per_Box promedio, Discount_Pct promedio y Marketing_Spend total.
Implementado (Sí/No)	Sí. Grupal.

Análisis de complejidad de cada uno de los pasos del algoritmo:

Pasos	**Complejidad**
get en pedidos_por_pais	O(1)
Recorrer los k pedidos del país y filtrar por fecha	O(k)
Por cada filtrado: get/put en la tabla de productos y actualizar acumulados	O(1)
value_set de la tabla de productos	O(p)
quick_sort de los p productos	O(p log p)
sub_list del top N y promedios	O(N)
TOTAL	***O(k + p log p)***

Análisis

Lo que se ordena son productos, no pedidos, y p es pequeño, así que en la práctica domina el recorrido O(k) de los pedidos del país. N no cambia el costo del filtro: siempre hay que acumular todos los pedidos antes de saber cuáles productos encabezan el ranking.

Requerimiento 6

Descripción

```python
def req_6(catalog, n, producto, fecha_ini, fecha_fin):
    """
    Retorna el resultado del requerimiento 6: los N paises con mayor eficiencia
    de mercadeo (suma Amount / suma Marketing_Spend) para un producto y un
    rango de fechas.

    Complejidad: O(1) para el get del producto + O(k) para filtrar y acumular
    por pais + O(c log c) para ordenar los c paises con merge_sort
    (c = numero de paises, pequeño y independiente de n).
    """
    start = get_time()
    paises, total_pedidos, amount_total, marketing_total = acumular_paises_req6(
        catalog, producto, fecha_ini, fecha_fin)

    # Solo entran al ranking los paises con Marketing_Spend total > 0
    acumulados = mlp.value_set(paises)
    ranking = al.new_list()
    for i in range(al.size(acumulados)):
        acumulador = al.get_element(acumulados, i)
        if acumulador["marketing"] > 0:
            acumulador["eficiencia"] = acumulador["amount"] / acumulador["marketing"]
            acumulador["precio_promedio"] = acumulador["suma_precio"] / acumulador["pedidos"]
            al.add_last(ranking, acumulador)

    ranking = al.merge_sort(ranking, criterio_req6)

    # Top N (si N es mayor a los paises disponibles, se muestran todos)
    top_n = al.new_list()
    i = 0
    while i < n and i < al.size(ranking):
        al.add_last(top_n, al.get_element(ranking, i))
        i += 1

    end = get_time()
    return {
        "tiempo_ms": delta_time(start, end),
        "total_pedidos": total_pedidos,
        "total_paises": mlp.size(paises),
        "amount_total": amount_total,
        "marketing_total": marketing_total,
```

Se obtiene de pedidos_por_producto la lista del producto y se recorre filtrando por rango de fechas. Por cada filtrado se suman los totales generales y se actualizan en una tabla auxiliar país → registro los acumulados del país (pedidos, Amount, Marketing_Spend, cajas y suma de precios). Luego se recorren los registros, se descartan los países con Marketing_Spend total igual a cero, se calcula la eficiencia (Amount / Marketing_Spend) y se ordenan con merge_sort por eficiencia descendente, Amount descendente y Country ascendente; se toman los primeros N.

Entrada	Catálogo, N, producto, fecha inicial y fecha final.
Salidas	Tiempo, total de filtrados, número de países distintos, Amount y Marketing_Spend totales y, por cada país del top N: pedidos, Amount, Marketing_Spend, cajas, Price_per_Box promedio y eficiencia de mercadeo.
Implementado (Sí/No)	Sí. Grupal.

Análisis de complejidad de cada uno de los pasos del algoritmo:

Pasos	**Complejidad**
get en pedidos_por_producto	O(1)
Recorrer los k pedidos del producto y filtrar por fecha	O(k)
Por cada filtrado: get/put en la tabla de países y actualizar acumulados	O(1)
value_set, descartar mercadeo cero y calcular eficiencia	O(c)
merge_sort de los c países	O(c log c)
sub_list del top N	O(N)
TOTAL	***O(k + c log c)***

Análisis

Igual que en el Req 5, se ordenan agregados y no pedidos: con solo cinco países el ordenamiento es despreciable y el costo lo define el recorrido de los pedidos del producto, O(k).

Resumen

Proceso	**Estructuras / ordenamiento**	**Complejidad**
Carga	Lista principal (quick_sort)	O(n log n)
Req 1	pedidos_por_mes + merge_sort	O(k + m log m)
Req 2	pedidos_por_pais + tabla de productos + quick_sort	O(k + m log m)
Req 3	pedidos_por_fecha + merge_sort	O(k + m log m)
Req 4	pedidos_por_producto_pais (sin ordenamiento)	O(k)
Req 5	pedidos_por_pais + tabla de productos + quick_sort de productos	O(k + p log p)
Req 6	pedidos_por_producto + tabla de países + merge_sort de países	O(k + c log c)

En todos los requerimientos la tabla de hash construida en la carga reemplaza el recorrido O(n) de la lista principal por un acceso O(1) a un subconjunto de k pedidos. El costo adicional se traslada a la carga, que se hace una sola vez.